

NODE ASSIGNMENT

STRUCTURING REST API

Creating a simple Api client

Assignment: Create a simple API client with functions to GET, POST, PUT, and DELETE. Instructions: • Create a JavaScript object that contains methods for each of the HTTP methods (GET, POST, PUT, DELETE)

Simple api client using api

```
Const apiClient = {
```

GET METHOD

```
Asyc get(url) {
```

```
Try {
```

```
Const response = await fetch(url);
```

```
Const data = await response.json();
```

```
Console.log("get response:", data);
```

```
Return data;
```

```
} catch (error) {
```

```
Console.log("get error:", error);
```

```
}
```

```
},
```

POST METHOD

```
Asyc post(url, newData) {
```

```
Try {
```

```
Const response = await fetch(url, {
```

```
Method: "post",
```

```
Headers: {
```

```
"content-type": "application/json"
```

```
},
```

```
Body: json.stringify(newdata)
```

```
});
```

```
Body: json.stringify(newData)}};
```

```

Const data = await response.json();
Console.log("post response:", data);
Return data;
}catch (error) {
Console.log("poat error:", error);
}},
PUT METHOD
Async put(url,updated data){
Try {
Const response = await fetch(url, {
Method: "put",
Headers: {
"content-type": "application/json"
},
Body: json.stringify(updateddata)
});
Const data = await response.json()
Console.log("put response: " , data);
Return data;
} catch (error) {
Console.log("put error:", error);
}},
DELETE METHOD
Async delete(url) {
Try {
Const response = await fetch(url, {
Method: "delete" });
Console.log("delete response:", response.status);
Return response.status;
}catch (error) {
Console.log( "delete error:" error)});}}};

```

Testing all methods

1. Get data

```
apiClient.get(https://jsonplaceholder.typicode.com/posts/1);
```

2 post data

```
apiClient.post("https://jsonplaceholder.typicode.com/posts",  
{  
  Title:"new post",  
  Body: "this is a new post",  
  userId: 1  
});
```

3 put (updated) data

```
apiClient.put("https://jsonplaceholder.typicode.com/posts/1",  
{  
  Id:1,  
  Title:"updated title",  
  Body: "updated body content",  
  Userid:1  
});
```

4 delete data

```
apiClient.data(  
  "https://jsonplaceholder.typicode.com/posts/1"  
);
```

Explanation

Get: fetch data from api

Post: add new data

Put: updated existing data

Delete: remove data

Fetch(): used to make http requests

Async.await: handles asynchronous API call easily

OUTPUT

```
PS C:\Users\lenovo\OneDrive\Desktop\work> nodemon index.jsx
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: jsx,json
[nodemon] starting `node index.jsx`
GET Response: {
  userId: 1,
  id: 1,
  title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit',
  body: 'quia et suscipit\n' +
    'suscipit recusandae consequuntur expedita et cum\n' +
    'reprehenderit molestiae ut ut quas totam\n' +
    'nostrum rerum est autem sunt rem eveniet architecto'
}
DELETE Response: 200
POST Response: { title: 'New Post', body: 'This is a new post', userId: 1, id: 101 }
PUT Response: {
  id: 1,
  title: 'Updated Title',
  body: 'Updated body content',
  userId: 1
}
[nodemon] clean exit - waiting for changes before restart
```

Pagination with API Data

Assignment:

Implement pagination for API data.

Instructions:

- Fetch data from the JSONPlaceholder API.
- Display only a limited number of posts (e.g., 5) at a time on the web page
- Create "Next" and "Previous" buttons to navigate between pages of results.

• Caching API Responses Assignment:

Implement basic caching for API responses.

Instructions:

- Fetch data from the JSONPlaceholder API.
- Store the response data in a variable or use local storage to cache the data
- Check if the data is cached before making a new request.

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 5000;
```

```
let cachedPosts = null;
```

```
const LIMIT = 5;
```

```
// Home Route
```

```
app.get("/", (req, res) => {  
  res.send("Server Running Successfully");  
});
```

```
// Posts Route
```

```
app.get("/posts", async (req, res) => {
```

```
  const page = parseInt(req.query.page) || 1;
```

```
  try {
```

```
    if (!cachedPosts) {
```

```
      const response = await fetch(  
        "https://jsonplaceholder.typicode.com/posts"  
      );
```

```
      const data = await response.json();
```

```
        cachedPosts = data;
    }

    const startIndex = (page - 1) * LIMIT;
    const endIndex = startIndex + LIMIT;

    const paginatedPosts =
        cachedPosts.slice(startIndex, endIndex);

    res.json({
        currentPage: page,
        posts: paginatedPosts
    });

} catch (error) {

    res.status(500).json({
        error: error.message
    });

}

});

app.listen(PORT, () => {
    console.log(`Server Running on http://localhost:${PORT}`);
});
```

OUTPUT

```
localhost:5000/posts?page=1
retty-print

{
  "currentPage": 1,
  "totalPosts": 100,
  "totalPages": 20,
  "posts": [
    {
      "userId": 1,
      "id": 1,
      "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
      "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
    },
    {
      "userId": 1,
      "id": 2,
      "title": "qui est esse",
      "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
    },
    {
      "userId": 1,
      "id": 3,
      "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
      "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
    },
    {
      "userId": 1,
      "id": 4,
      "title": "eum et est occaecati",
      "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iture\nquis sunt voluptatem rerum illo velit"
    },
    {
      "userId": 1,
      "id": 5,
      "title": "nesciunt voluptate doloribus omnis",
      "body": "vero autem doloribus doloribus omnis\nvoluptatem blanditiis blanditiis eveniet eos aut animi\nid est eius vero doloribus doloribus\nomnis vero voluptate et iusto placeat deserunt sunt\nquia id ut eius voluptate et omnis\nquia et iusto placeat deserunt sunt vel qui est odio et odit aut nostrum et illo vel qui\nautem rerum non est autem asperiores aut autem est amet amet"
    }
  ]
}
```

```
localhost:5000/posts?page=2
retty-print

{
  "currentPage": 2,
  "totalPosts": 100,
  "totalPages": 20,
  "posts": [
    {
      "userId": 1,
      "id": 6,
      "title": "dolorem eum magni eos aperiam quia",
      "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et doloremque molestiae"
    },
    {
      "userId": 1,
      "id": 7,
      "title": "magnam facilis autem",
      "body": "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
    },
    {
      "userId": 1,
      "id": 8,
      "title": "dolorem dolore est ipsam",
      "body": "dignissimos aperiam dolorem qui eum\nfacilis quibusdam animi sint suscipit qui sint possimus cum\nquaerat magni maiores excepturi\nipsam ut commodi dolor voluptatum modi aut vitae"
    },
    {
      "userId": 1,
      "id": 9,
      "title": "nesciunt iure omnis dolorem tempora et accusantium",
      "body": "consectetur animi nesciunt iure dolore\nenim quia ad\nveniam autem ut quam aut nobis\net est aut quod aut provident voluptas autem voluptas"
    },
    {
      "userId": 1,
      "id": 10,
      "title": "optio molestias id quia eum",
      "body": "quo et expedita modi cum officia vel magni\ndoloribus qui repudiandae\nvero nisi sit\nloquas veritatis quod sed accusamus veritatis error"
    }
  ]
}
```

HTTP AND WEB SERVERS

Theory Assignment

What is http?

http stands for hypertext transfer protocol

it is used for communication between

client/browser and server

ex:

user opens browser

browser sends request to server

server sends response back

how node.js handles http requests and responses?

Node uses the built-in http modules to create web server

When client sends a request

Node receives the request

Processes the request

Sends a response back to the client

Request-response lifecycle

Browser>request> node.js server > process request>response>browser

Detailed lifecycle

1) Client sends request

The browser sends an http request to the server

2) Server receives request

Node server receives:

Request method(get, post)

url

header

data

ex:

req.method

req.url

3) Server Processes Request

The server checks:

Which route is requested

What operation should be performed

Example:

if(req.url === "/")

4. Server Sends Response

Server sends:
Status code
Headers
Data/message

Example:
res.write("Hello World");
res.end();

5. Browser Displays Response

The browser receives the response and shows it to the user.

Important HTTP Methods

Method	Purpose
GET	Fetch data
POST	Send data
PUT	Update data
DELETE	Remove data

Important HTTP Status Codes

Status Code Meaning

200	Success
404	Not Found
500	Server Error

Practical

Create a simple http server

Server.js

```
const http = require("http");
```

```
// Create Server
```

```
const server = http.createServer((req, res) => {
```

```
// Check request method
if (req.method === "GET") {

  // Response Header
  res.writeHead(200, {
    "Content-Type": "text/plain"
  });

  // Send Response
  res.end("Hello World");
}

});

// Port Number
const PORT = 3000;

// Start Server
server.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

OUTPUT



HANDLING DIFFERENT URL ROUTES IN NODE.JS

Assignment

Create an http server that handles different url paths

```
const http = require("http");
```

```
// Create Server
```

```
const server = http.createServer((req, res) => {
```

```
// Log Requested URL
console.log("Requested URL:", req.url);

// Home Route
if (req.url === "/") {

    res.writeHead(200, {
        "Content-Type": "text/plain"
    });

    res.end("Welcome to the Home Page!");

}

// About Route
else if (req.url === "/about") {

    res.writeHead(200, {
        "Content-Type": "text/plain"
    });

    res.end("This is the About Page.");

}

// Contact Route
else if (req.url === "/contact") {

    res.writeHead(200, {
        "Content-Type": "text/plain"
```

```
});
```

```
res.end("This is the Contact Page.");
```

```
}
```

```
// 404 Route
```

```
else {
```

```
res.writeHead(404, {
```

```
  "Content-Type": "text/plain"
```

```
});
```

```
res.end("404 Page Not Found");
```

```
}
```

```
});
```

```
// Port Number
```

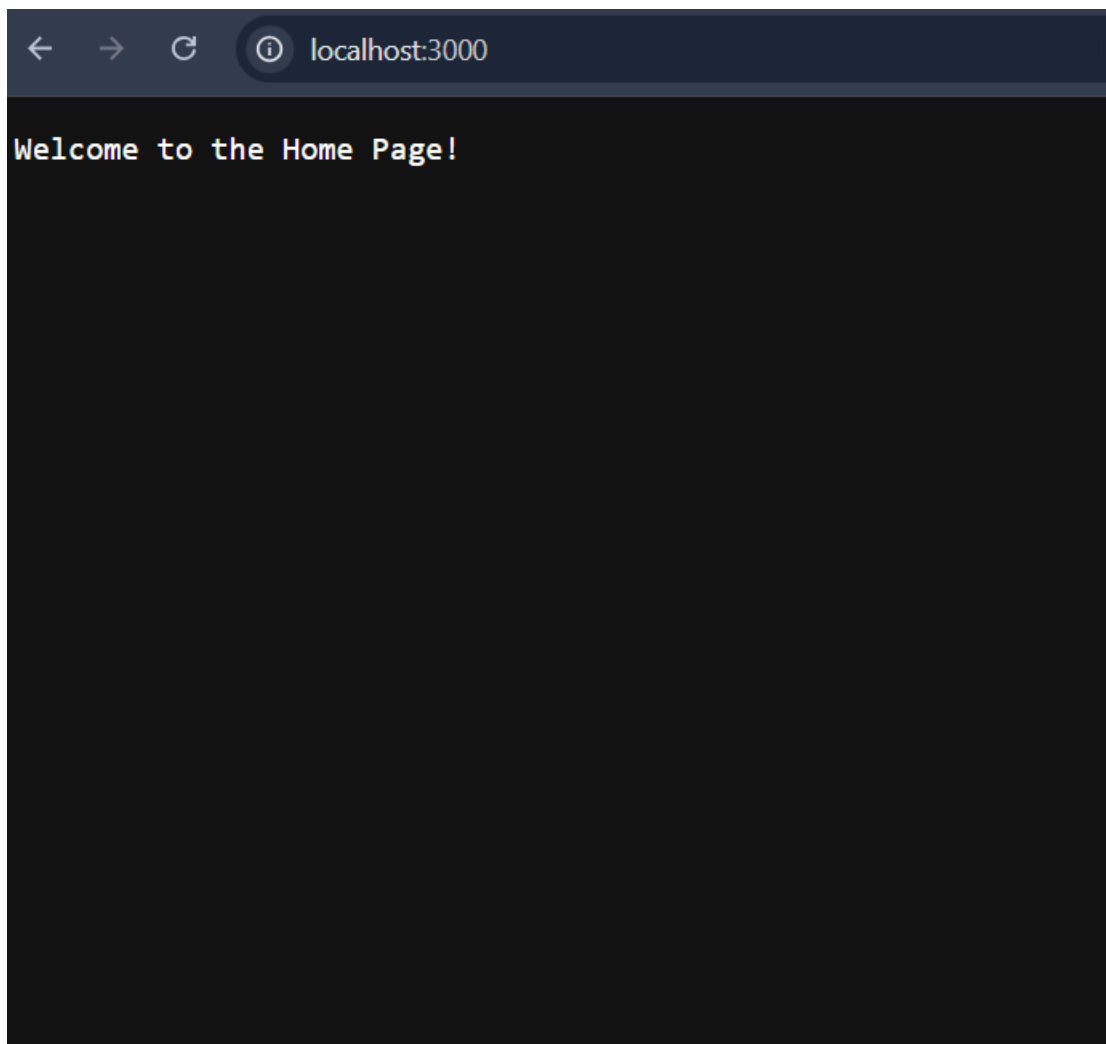
```
const PORT = 3000;
```

```
// Start Server
```

```
server.listen(PORT, () => {
```

```
  console.log(`Server running at http://localhost:${PORT}`);
```

```
});
```



HANDLING HTTP GET REQUEST

Create `/api/users` route that returns json data

```
const http = require("http");
```

```
// Sample User Data
```

```
const users = [  
  {  
    id: 1,  
    name: "Devika",  
    email: "devika@gmail.com"  
  },  
]
```

```
{
  id: 2,
  name: "Rahul",
  email: "rahul@gmail.com"
},
{
  id: 3,
  name: "Priya",
  email: "priya@gmail.com"
}
];

// Create Server
const server = http.createServer((req, res) => {

  console.log("Requested URL:", req.url);

  // Handle GET Request
  if (req.method === "GET" && req.url === "/api/users") {

    // Set Header
    res.setHeader("Content-Type", "application/json");

    // Send JSON Response
    res.end(JSON.stringify(users));

  }

  else {

    res.writeHead(404, {
```

```
      "Content-Type": "text/plain"
    });

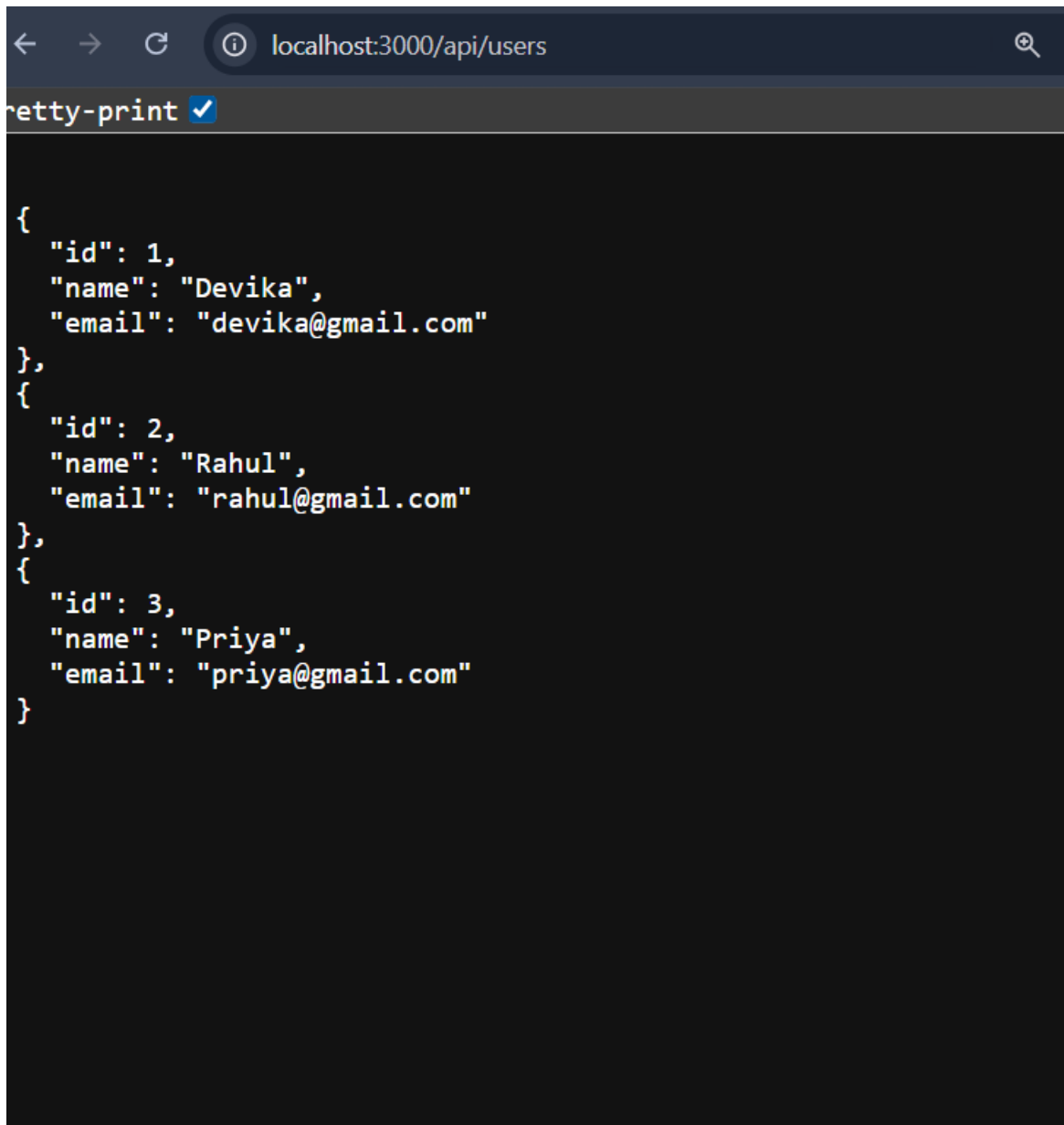
    res.end("Route Not Found");

  }

});

// Port Number
const PORT = 3000;

// Start Server
server.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```


A screenshot of a web browser window. The address bar shows 'localhost:3000/api/users'. Below the address bar, there is a dark grey bar with the text 'pretty-print' and a blue checkmark icon. The main content area of the browser displays a JSON array of three user objects. The first object has 'id': 1, 'name': 'Devika', and 'email': 'devika@gmail.com'. The second object has 'id': 2, 'name': 'Rahul', and 'email': 'rahul@gmail.com'. The third object has 'id': 3, 'name': 'Priya', and 'email': 'priya@gmail.com'.

```
{
  "id": 1,
  "name": "Devika",
  "email": "devika@gmail.com"
},
{
  "id": 2,
  "name": "Rahul",
  "email": "rahul@gmail.com"
},
{
  "id": 3,
  "name": "Priya",
  "email": "priya@gmail.com"
}
```

Implement handling for post requests using node.js and body-parser

```
const express = require("express");
```

```
const bodyParser = require("body-parser");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Middleware
```

```
app.use(bodyParser.json());
```

```
// Home Route
```

```
app.get("/", (req, res) => {  
  res.send("Server is Running");  
});
```

```
// POST Route
```

```
app.post("/api/users", (req, res) => {
```

```
  const userData = req.body;
```

```
  console.log("Received Data:", userData);
```

```
  res.json({  
    message: "User data received successfully",  
    data: userData  
  });
```

```
});
```

```
// Start Server
```

```
app.listen(PORT, () => {  
  console.log(`Server running on http://localhost:${PORT}`);  
});
```



SERVING STATIC FILES USING NODE.JS

Server html,css, and javascript files using

http

fs

path

public/index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Static Files</title>
```

```
  <link rel="stylesheet" href="/style.css">
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome to Node.js Static Server</h1>
```

```
  <button onclick="showMessage()">
```

```
    Click Me
```

```
</button>
```

```
<script src="/script.js"></script>
```

```
</body>
```

```
</html>
```

Public/style.css

```
body{
```

```
    background-color: lightblue;
```

```
    text-align: center;
```

```
    font-family: Arial;
```

```
}
```

```
h1{
```

```
    color: darkblue;
```

```
}
```

Public/script.js

```
function showMessage() {
```

```
    alert("JavaScript File Loaded Successfully");
```

```
}
```

Server.js

```
const http = require("http");
```

```
const fs = require("fs");
```

```
const path = require("path");
```

```
const PORT = 3000;
```

```
// Create Server
```

```
const server = http.createServer((req, res) => {
```

```
let filePath = "";

// Home Route
if (req.url === "/") {

    filePath = path.join(__dirname, "public", "index.html");

    res.setHeader("Content-Type", "text/html");

}

// CSS File
else if (req.url === "/style.css") {

    filePath = path.join(__dirname, "public", "style.css");

    res.setHeader("Content-Type", "text/css");

}

// JavaScript File
else if (req.url === "/script.js") {

    filePath = path.join(__dirname, "public", "script.js");

    res.setHeader("Content-Type", "application/javascript");

}

// 404
else {
```

```
res.writeHead(404, {
  "Content-Type": "text/plain"
});

res.end("404 File Not Found");

return;

}

// Read and Serve File
fs.readFile(filePath, (err, data) => {

  if (err) {

    res.writeHead(500, {
      "Content-Type": "text/plain"
    });

    res.end("Server Error");

  }

  else {

    res.writeHead(200);

    res.end(data);

  }

}
```

```
});
```

```
});
```

```
// Start Server
```

```
server.listen(PORT, () => {  
  console.log(`Server running on http://localhost:${PORT}`);  
});
```

CREATING A SIMPLE RESTFUL API IN NODE.JS

Create a restful that performs:

Get

Post

Put

Delete

Operations using an in-memory array

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Middleware
```

```
app.use(express.json());
```

```
// In-Memory Array
```

```
let items = [  
  {  
    id: 1,  
    name: "Laptop"
```

```
    },  
    {  
      id: 2,  
      name: "Mobile"  
    },  
    {  
      id: 3,  
      name: "Keyboard"  
    }  
  ];
```

```
// GET All Items
```

```
app.get("/api/items", (req, res) => {
```

```
  try {
```

```
    res.json(items);
```

```
  } catch (error) {
```

```
    console.log(error.message);
```

```
    res.status(500).json({  
      message: "Server Error"  
    });
```

```
  }
```

```
});
```



```
// POST Add New Item
app.post("/api/items", (req, res) => {

  try {

    const newItem = {
      id: items.length + 1,
      name: req.body.name
    };

    items.push(newItem);

    res.status(201).json({
      message: "Item Added Successfully",
      data: newItem
    });

  } catch (error) {

    console.log(error.message);

    res.status(500).json({
      message: "Server Error"
    });

  }

});
```

```
// PUT Update Item
app.put("/api/items/:id", (req, res) => {

  try {

    const itemId = parseInt(req.params.id);

    const item = items.find((i) => i.id === itemId);

    // Item Not Found
    if (!item) {

      return res.status(404).json({
        message: "Item Not Found"
      });

    }

    // Update Item
    item.name = req.body.name;

    res.json({
      message: "Item Updated Successfully",
      data: item
    });

  } catch (error) {

    console.log(error.message);

    res.status(500).json({
```

```
        message: "Server Error"
    });

}

});

// DELETE Item
app.delete("/api/items/:id", (req, res) => {

    try {

        const itemId = parseInt(req.params.id);

        const itemIndex = items.findIndex((i) => i.id === itemId);

        // Item Not Found
        if (itemIndex === -1) {

            return res.status(404).json({
                message: "Item Not Found"
            });

        }

        // Delete Item
        const deletedItem = items.splice(itemIndex, 1);

        res.json({
            message: "Item Deleted Successfully",
```

```
        data: deletedItem
    });

    } catch (error) {

        console.log(error.message);

        res.status(500).json({
            message: "Server Error"
        });

    }

});

// 404 Middleware
app.use((req, res) => {

    console.log("404 Error: Route Not Found");

    res.status(404).json({
        message: "Page not found!"
    });

});

// Server Error Middleware
app.use((err, req, res, next) => {
```

```
console.log("Server Error:", err.message);

res.status(500).json({
  message: "Internal Server Error"
});

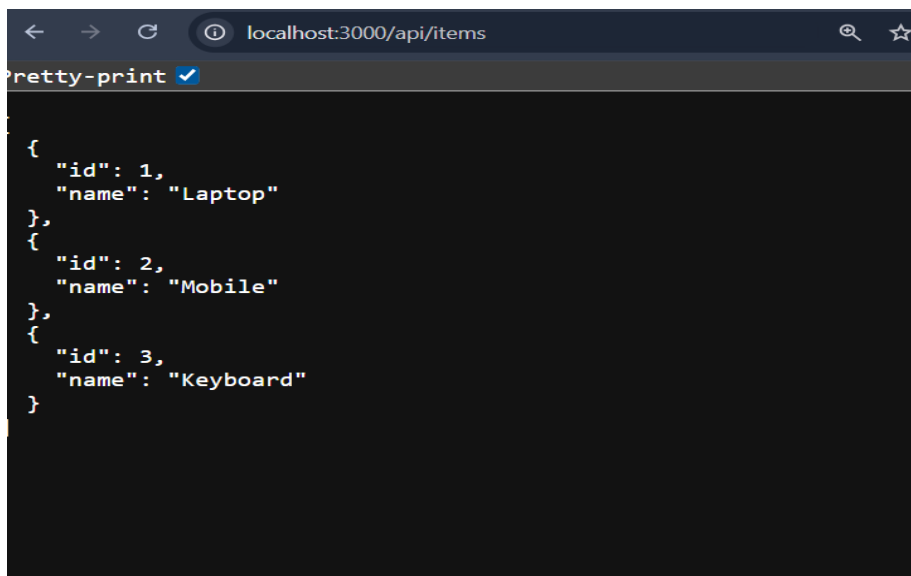
});

// Start Server
app.listen(PORT, () => {

  console.log(`Server running on http://localhost:${PORT}`);

});
```

get all items



USING QUERY PARAMETERS IN NODE.JS

Handle query parameters in http requests

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// GET Route with Query Parameter
```

```
app.get("/api/search", (req, res) => {
```

```
    // Get Query Parameter
```

```
    const searchQuery = req.query.q;
```

```
    // Log Query
```

```
    console.log("Query Parameter:", req.query);
```

```
    // Send Response
```

```
    res.json({
```

```
        query: searchQuery,
```

```
        message: `You searched for: ${searchQuery}`
```

```
    });
```

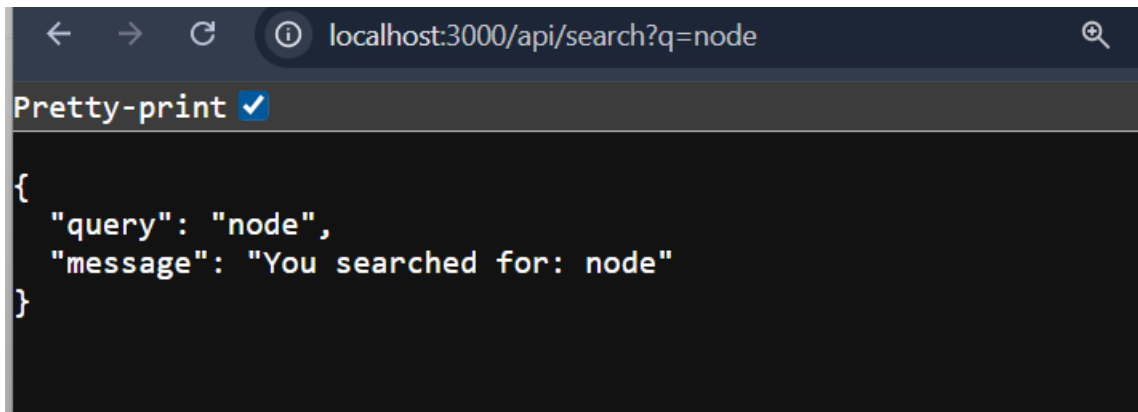
```
});
```

```
// Start Server
```

```
app.listen(PORT, () => {
```

```
    console.log(`Server running on http://localhost:${PORT}`);
```

```
});
```



```
localhost:3000/api/search?q=node

Pretty-print ✓

{
  "query": "node",
  "message": "You searched for: node"
}
```

Query parameters:

Query parameters are values passed in url

Ex: ?q=node

Q: parameter name

Node: parameter value

Access query parameters: req.query.q

Gets value of query parameter

Creating a basic form and handling form submission

Public/index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Form Submission</title>
```

```
</head>
```

```
<body>
```

```
  <h2>User Form</h2>
```

```
  <form action="/submit" method="POST">
```

```
    <input
```

```
    type="text"
    name="name"
    placeholder="Enter Name"
    required
  >
```

```
<br><br>
```

```
<input
  type="email"
  name="email"
  placeholder="Enter Email"
  required
>
```

```
<br><br>
```

```
<button type="submit">
  Submit
</button>
```

```
</form>
```

```
</body>
```

```
</html>
```

```
Server.js
```

```
const express = require("express");
```

```
const path = require("path");
```

```
const app = express();
```



```
const PORT = 3000;

// Middleware to Parse Form Data
app.use(express.urlencoded({ extended: true }));

// Serve HTML Form
app.get("/", (req, res) => {

    res.sendFile(path.join(__dirname, "public", "index.html"));

});

// Handle Form Submission
app.post("/submit", (req, res) => {

    // Get Form Data
    const { name, email } = req.body;

    console.log("Form Data:", req.body);

    // Send Response
    res.send(`
        <h2>Form Submitted Successfully</h2>

        <p>Name: ${name}</p>
        <p>Email: ${email}</p>
    `);
}
```

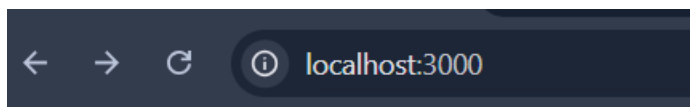
```
});
```

```
// Start Server
```

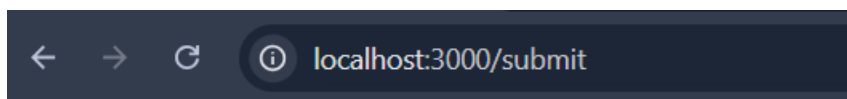
```
app.listen(PORT, () => {
```

```
  console.log(`Server running on http://localhost:${PORT}`);
```

```
});
```



User Form



Form Submitted Successfully

Name: devika

Email: kollurdevika@gmail.com

IMPLEMENTING MIDDLEWe

Create middleware that logs

Request method

Request url

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Custom Middleware
```

```
const loggerMiddleware = (req, res, next) => {
```

```
  console.log(
```

```
    `Method: ${req.method} | URL: ${req.url}`
```

```
  );
```

```
  next();
```

```
};
```

```
// Use Middleware
```

```
app.use(loggerMiddleware);
```

```
// Routes
```

```
app.get("/", (req, res) => {
```

```
  res.send("Home Page");
```

```
});
```

```
app.get("/about", (req, res) => {
```

```
    res.send("About Page");
```

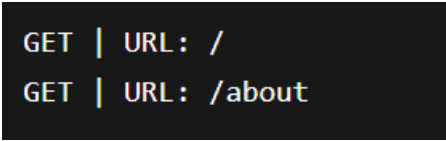
```
});
```

```
// Start Server
```

```
app.listen(PORT, () => {
```

```
    console.log(`Server running on http://localhost:${PORT}`);
```

```
});
```



```
GET | URL: /  
GET | URL: /about
```

Explanation of middleware

What is middleware?

Middleware is a function that runs:

- Before request reaches route

- Between request and response

Middleware syntax : (req, res, next)

- Req: request object

- Res: response object

- Next: moves to next middleware/route

Why use middleware?

Used for:

- Logging validation error handling

- Authentication parsing data

BASIC AUTHENTICATION IN NODE.JS

Implement:

Login route

Username/password validation

Protected route access

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Middleware
```

```
app.use(express.json());
```

```
// Dummy User Data
```

```
const USERNAME = "admin";
```

```
const PASSWORD = "12345";
```

```
// Login Status
```

```
let isLoggedIn = false;
```

```
// Login Route
```

```
app.post("/login", (req, res) => {
```

```
    const { username, password } = req.body;
```

```
    // Validate Credentials
```

```
    if (
```

```
        username === USERNAME &&
```

```
    password === PASSWORD
  ) {

    isLoggedIn = true;

    res.json({
      message: "Login Successful"
    });

  }

  else {

    res.status(401).json({
      message: "Unauthorized User"
    });

  }

});

// Protected Route
app.get("/api/protected", (req, res) => {

  if (isLoggedIn) {

    res.json({
      message: "Welcome to Protected Route"
    });

  }

});
```

```

    }

    else {

        res.status(401).json({
            message: "Please login first"
        });

    }

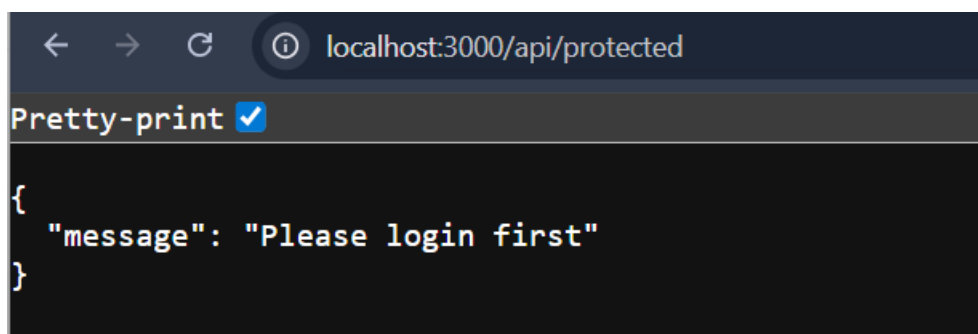
});

// Start Server
app.listen(PORT, () => {

    console.log(`Server running on http://localhost:${PORT}`);

});

```



Setting custom header

Send custom header in response

```
const http = require("http");
```

```
const server = http.createServer((req, res) => {
```

```
// Custom Headers
res.setHeader("X-Powered-By", "Node.js");

res.setHeader(
  "Content-Type",
  "text/plain"
);

// Log Headers
console.log(res.getHeaders());

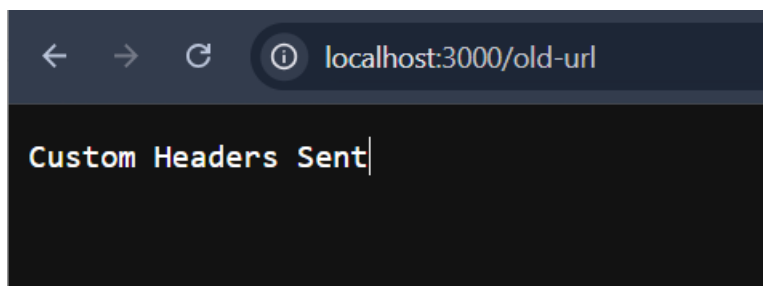
// Response
res.end("Custom Headers Sent");

});

server.listen(3000, () => {

  console.log("Server running on port 3000");

});
```




```
C:\Users\lenovo\OneDrive\Desktop\work>node server.js
Server running on port 3000
[Object: null prototype] {
  'x-powered-by': 'Node.js',
  'content-type': 'text/plain'
}
[Object: null prototype] {
  'x-powered-by': 'Node.js',
  'content-type': 'text/plain'
}
```

Rate limiting middleware

Limit request

5 request

Per minute

```
const express = require("express");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// In-Memory Store
```

```
const requestCounts = {};
```

```
// Rate Limit Middleware
```

```
const rateLimiter = (req, res, next) => {
```

```
  const userIP = req.ip;
```

```
  // Create User Entry
```

```
  if (!requestCounts[userIP]) {
```

```
    requestCounts[userIP] = {
```

```
        count: 1,
        startTime: Date.now()
    };

}

else {

    const currentTime = Date.now();

    const timeDifference =
        currentTime - requestCounts[userIP].startTime;

    // Reset After 1 Minute
    if (timeDifference > 60000) {

        requestCounts[userIP] = {
            count: 1,
            startTime: currentTime
        };

    }

    else {

        requestCounts[userIP].count++;

    }

}
```

```
// Limit Check
if (requestCounts[userIP].count > 5) {

    return res.status(429).json({
        message: "Too many requests. Try again later."
    });

}

next();

};

// Use Middleware
app.use(rateLimiter);

// Route
app.get("/", (req, res) => {

    res.send("Welcome to Rate Limited Server");

});

// Start Server
app.listen(PORT, () => {

    console.log(`Server running on http://localhost:${PORT}`);
```

```
});
```

Pretty-print ☒

```
{  
  "message": "Too many requests. Try again later."  
}
```

Setting up a proxy

Assignment

Proxy request to external api using : http-proxy-middleware

```
const express = require("express");
```

```
const {
```

```
  createProxyMiddleware
```

```
} = require("http-proxy-middleware");
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Logger Middleware
```

```
app.use((req, res, next) => {
```

```
  console.log(
```

```
    `Request: ${req.method} ${req.url}`
```

```
  );
```

```
  next();
```

```
});
```

```
// Proxy Route
```

```
app.use(
```

```
  "/posts",
```

```
  createProxyMiddleware({
```

```
    target:
```

```
    "https://jsonplaceholder.typicode.com",
```

```
    changeOrigin: true
```

```
  })
```

```
);
```

```
// Start Server
```

```
app.listen(PORT, () => {
```

```
  console.log(
```

```
    `Server running on http://localhost:${PORT}`
```

```
  );
```

```
});
```

```
localhost:3000/posts
Pretty-print ☒

{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt re eveniet architecto"
},
{
  "userId": 1,
  "id": 2,
  "title": "qui est esse",
  "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor bea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
},
{
  "userId": 1,
  "id": 3,
  "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
  "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius o labore et velit aut"
},
{
  "userId": 1,
  "id": 4,
  "title": "eum et est occaecati"
```